

# FinResearch AI

What the project does, in plain language

FinResearch AI is a web application that reads company financial documents for you and answers questions about them. You upload reports such as annual reports, quarterly results or earnings-call transcripts, ask a question in normal English, and get an answer that shows exactly which page it came from.

**One-sentence summary:** it is a research assistant for financial documents that retrieves the relevant passages, calculates the financial ratios itself in code, and writes an answer with page-level citations so every claim can be checked.

## 1. The problem it solves

An annual report can be 200 pages long. Finding why profit fell, what the debt level is, or what management expects next year means searching through text, tables and footnotes by hand. General AI chatbots can help, but they have three weaknesses with financial documents:

- **They make arithmetic mistakes.** Asking a language model to compute a ratio often gives a wrong number.
- **They invent facts.** A model may state a figure or a source that does not exist in the document.
- **They break tables.** Simple tools cut documents into fixed-size pieces, splitting a balance sheet in half so its numbers lose their meaning.

This project is designed specifically to avoid those three failures.

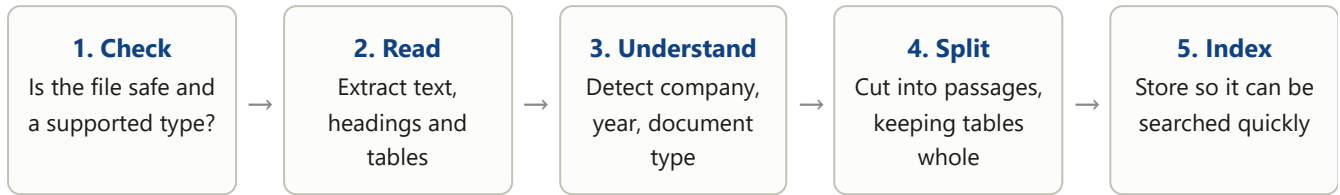
## 2. What you can do with it

| Feature            | What it does                                                                                                                                            |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Upload documents   | Accepts PDF, Word, Excel and text files. It automatically detects the company, the type of document, the financial year and the currency.               |
| Research Chat      | Ask questions such as "Why did operating margins decline?" and get an answer with clickable source citations. Follow-up questions remember the context. |
| Financial Analysis | Shows key figures (revenue, profit, EPS), 18 financial ratios, trend charts and automatic risk warnings for a company.                                  |
| Comparisons        | Compares two companies side by side, or one company across different years.                                                                             |
| Reports            | Generates a full 13-section research report (revenue, profitability, cash flow, risks, and so on) with a list of sources.                               |
| Dashboard          | Overview of companies, documents, recent questions and system performance.                                                                              |

### 3. How it works, step by step

---

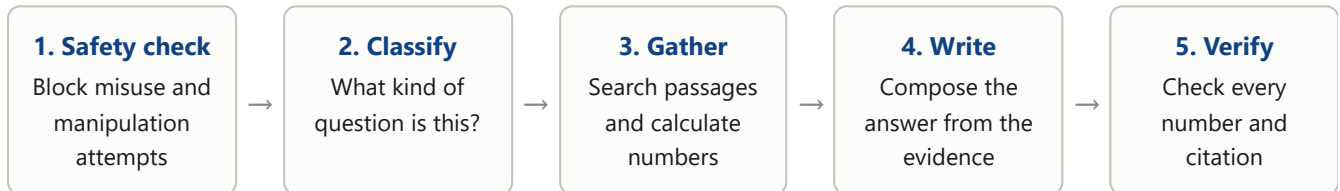
#### Step A: when you upload a document



Two things are saved from every document:

- **Passages of text** (about a paragraph each), stored in a searchable index along with their page number and section name.
- **Financial facts** taken from the tables, for example "Revenue, FY2025, 10,600, INR crore, page 4". These are stored as structured data, like rows in a spreadsheet.

#### Step B: when you ask a question



1. **Safety check.** Requests such as "ignore your instructions" or "how do I manipulate a share price" are refused. Questions asking for guaranteed returns get a clear statement that no such guarantee is possible.
2. **Classify the question.** The system decides which of nine types it is (a calculation, a risk question, a comparison, a summary, a trend, and so on). This decides which specialist components handle it.
3. **Gather evidence.** Two searches run together: one by meaning (finds passages about the same idea even with different words) and one by keywords (finds exact terms). The results are merged and re-ranked so only the 6 to 8 best passages are kept. If the question involves numbers, a calculation engine computes them from the stored financial facts.
4. **Write the answer.** A language model writes the answer using only the gathered passages and the pre-calculated numbers. It is instructed never to do arithmetic or add outside facts.
5. **Verify.** After writing, the answer is checked automatically. Any citation that does not match a real source is removed, and every figure in the answer is compared with the evidence. The result is shown as a "Grounded" or "Partially grounded" badge.

**The key design rule:** the language model only writes sentences. It never calculates. All numbers come from Python code that is covered by automated tests, and each number can be traced to the page it was read from.

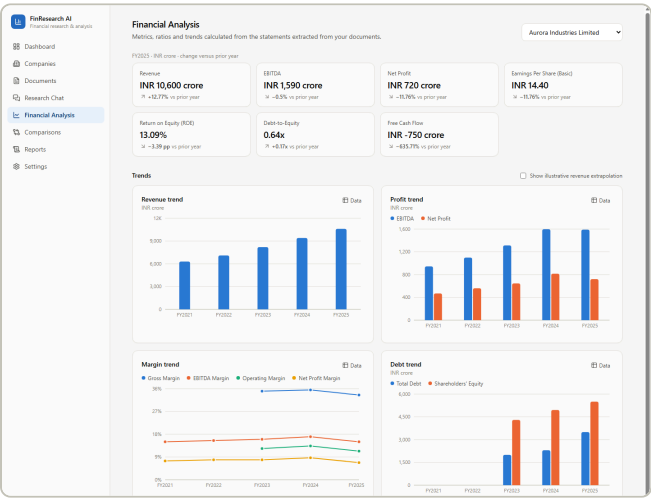
## 4. A worked example

**Question:** "What is Aurora's debt-to-equity ratio?"

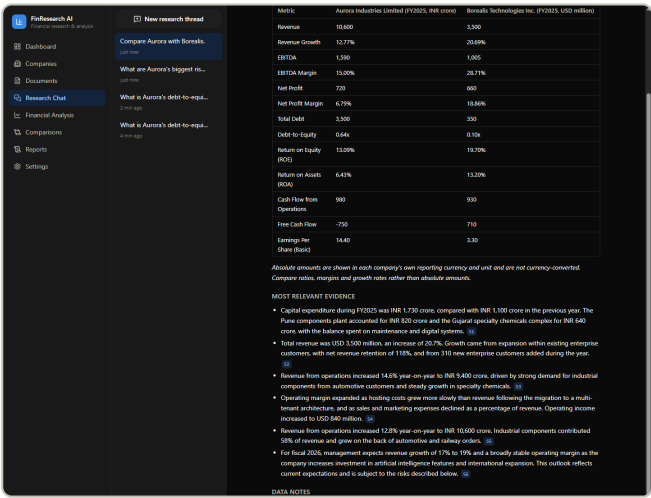
| What the system does      | Result                                                                                                                                        |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Classifies the question   | Financial calculation                                                                                                                         |
| Looks up stored facts     | Long-term borrowings 2,600 + short-term borrowings 900 = total debt 3,500. Shareholders' equity 5,500. Both from the balance sheet on page 5. |
| Calculates in code        | $3,500 \div 5,500 = \mathbf{0.64x}$                                                                                                           |
| Retrieves supporting text | Management's statement that it intends to bring the ratio below 0.5 by FY2027, plus the balance sheet table itself                            |
| Shows the answer          | The ratio, the formula, each input with a link to its source page, and the supporting passages                                                |

If a number needed for a calculation is missing from the documents, the system says "Not available" rather than estimating it.

## 5. What it looks like



Financial Analysis: key figures, trend charts and ratios



Research Chat: a company comparison with citations

## 6. The main parts of the system

| Part               | Job                                                                                                        | Built with                                      |
|--------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Website (frontend) | The pages you see and click on                                                                             | Next.js, React, TypeScript                      |
| Server (backend)   | Receives requests and runs the logic                                                                       | Python, FastAPI                                 |
| Document reader    | Extracts text and tables from files                                                                        | PyMuPDF, pdfplumber                             |
| Search             | Finds relevant passages by meaning and by keyword                                                          | BGE embeddings, BM25, a re-ranking model        |
| Agents             | Specialist components for research, calculation, comparison, risk and writing, coordinated by a supervisor | LangGraph                                       |
| Calculation engine | Ratios, growth rates, trends, risk signals                                                                 | Plain Python                                    |
| Language model     | Writes the final answer in readable English                                                                | Any provider: Groq, OpenAI or a local model     |
| Storage            | Documents, passages, facts, conversations                                                                  | SQLite or PostgreSQL; Qdrant for search vectors |

"RAG" (Retrieval-Augmented Generation) is the name for this overall approach: first *retrieve* relevant text from your own documents, then let the model *generate* an answer from that text only.

## 7. Where the data comes from

The application only knows what is in the documents you upload. It does not download market data or company information from the internet.

**About the demo data:** the two companies currently loaded, "Aurora Industries Limited" and "Borealis Technologies Inc.", are fictitious. Their reports were generated by a script in this project purely so the system can be demonstrated and tested. None of their figures describe a real business. To analyse real companies, upload their actual reports on the Documents page.

## 8. Current state and limits

- **Tested and working on this computer:** uploading, searching, calculations, comparisons, reports and the full website. 341 automated tests pass.
- **Running without a language model right now:** no API key is configured, so answers are "extractive", meaning they show the calculated numbers and quote the most relevant passages directly instead of writing new prose. Adding a key (for example from Groq) switches on written answers.
- **Not yet run:** the Docker setup, the PostgreSQL database, a real Qdrant server and a live language-model provider. The code for these exists but was only checked with simulated connections.

- **Table reading is rule-based.** It works on standard statement layouts. Unusual or scanned tables may not be read, in which case ratios show "Not available" while text questions still work.
- **It is a research aid, not a financial advisor.** It does not recommend buying or selling anything, and its output should be checked against the cited pages.